

二分查找在实际解题当中的运用

由于笔者最近在刷题的过程当中连续遇到了多道不同的可以使用二分查找大大优化算法效率的题目，索性直接写一个总结文档，把有关于二分查找常见的问题做一个统一分析，刚好也可以作为南京大学高级算法课程的一个课程作业

本文当中使用到的题目以及链接

[LeetCode 2300 Successful Pairs of Spells and Potions](#)

[LeetCode 1292 Maximum Side Length of a Square with Sum Smaller Than Threshold](#)

[LeetCode 3350 Adjacent Increasing Subarrays Detection II](#)

[LeetCode 875 Koko Eating Bananas](#)

[LeetCode 410 Split Array Largest Sum](#)

概述

二分查找不是“在数组里找数”，本质是**在一个单调区间中寻找边界**

也就是说，我们利用这样一种结构：

TTTTTTTTTTTTFFFFF

当某个条件从 **True** 变成 **False** 时，就存在一个**边界点**，二分查找就是寻找这个边界

时间复杂度： $O(\log n)$

二分查找的三种核心类型

有序数组二分

典型例题：

[LeetCode 2300 Successful Pairs of Spells and Potions](#)

2300. 咒语和药水的成功对数

中等 相关标签 相关企业 提示 Aa

给你两个正整数数组 `spells` 和 `potions`，长度分别为 `n` 和 `m`，其中 `spells[i]` 表示第 `i` 个咒语的能量强度，`potions[j]` 表示第 `j` 瓶药水的能量强度。

同时给你一个整数 `success`。一个咒语和药水的能量强度 **相乘** 如果 **大于等于** `success`，那么它们视为一对 **成功** 的组合。

请你返回一个长度为 `n` 的整数数组 `pairs`，其中 `pairs[i]` 是能跟第 `i` 个咒语成功组合的 **药水** 数目。

这一题是较为经典的对于数组的二分，由于题目给出的直接操作对象就是数组，我们也可以非常容易看出实际上对于每一个药水，我们只需要首先对于咒语的能量进行排序，使用显式的二分查找找到第一个满足条件的咒语，将余下的咒语数量全部加入 `ans` 当中即可

本题单调对象：天然的数组排序

题解：

```
3 import java.util.Arrays;
4
5 public class NicePairs_2300 { 0个用法 吕 hasn <d>
6     public int[] successfulPairs(int[] spells, int[] potions, long success) { 0个用法 吕 hasn <d>
7         Arrays.sort(potions);
8
9         int m = spells.length, n = potions.length;
10
11         int[] count = new int[m];
12
13         for(int i = 0; i < m; i++){
14             int left = 0, right = n - 1;
15             int pos = n;
16
17             while(left <= right){
18                 int mid = left + (right - left) / 2;
19
20                 if((long)spells[i] * potions[mid] >= success){
21                     pos = mid;
22                     right = mid - 1;
23                 } else {
24                     left = mid + 1;
25                 }
26             }
27
28             count[i] = n - pos;
29         }
30
31         return count;
32     }
33 }
```

答案二分

典型例题：

[LeetCode 1292 Maximum Side Length of a Square with Sum Smaller Than Threshold](#)

1292. 元素和小于等于阈值的正方形的最大边长

已解答

中等 相关标签 相关企业 提示 Az

给你一个大小为 $m \times n$ 的矩阵 mat 和一个整数阈值 $threshold$ 。

请你返回元素总和小于或等于阈值的正方形区域的最大边长；如果没有这样的正方形区域，则返回 0 。

示例 1：

1	1	3	2	4	3	2
1	1	3	2	4	3	2
1	1	3	2	4	3	2

输入：mat = [[1,1,3,2,4,3,2],[1,1,3,2,4,3,2],[1,1,3,2,4,3,2]], threshold = 4
输出：2
解释：总和小于或等于 4 的正方形的最大边长为 2，如图所示。

本题要求的最后结果是符合题意的正方形的最大边长，所以存在一个分界点 `spilt`，当边长大于这个 `spilt` 的时候，无论如何不会符合条件，反之必然符合条件

构成了先前提到的 **TTTTTTFFFF** 结构，于是我们可以对于答案的范围进行二分查找，来快速定位（对数复杂度）

本题单调对象：答案合法性

题解

```
3 public class MaxSideLength_1292 { 0 个用法 吕 hasn <d> *
4
5     int[][] prefixSum; 9 个用法
6
7     public int maxSideLength(int[][] mat, int threshold) { 0 个用法 吕 hasn <d> *
8         int m = mat.length, n = mat[0].length;
9         prefixSum = new int[m + 1][n + 1];
10        // 构建二维前缀和
11        for(int i = 1; i <= m; i++){
12            for(int j = 1; j <= n; j++){
13                prefixSum[i][j] = mat[i - 1][j - 1]
14                    + prefixSum[i - 1][j]
15                    + prefixSum[i][j - 1]
16                    - prefixSum[i - 1][j - 1];
17            }
18        }
19        int left = 0, right = Math.min(m, n);
20        while(left < right){
21            int mid = left + (right - left + 1) / 2;
22            if(check(mat, mid, threshold)){
23                left = mid;
24            }else{
25                right = mid - 1;
26            }
27        }
28        return left;
29    }
```

```

31 @ private boolean check(int[][] mat, int length, int threshold){ 1 个用法 吕 hasn <d> *
32     for(int i = 1; i <= mat.length - length + 1; i++){
33         for(int j = 1; j <= mat[0].length - length + 1; j++){
34             int sum =
35                 prefixSum[i + length - 1][j + length - 1]
36                 - prefixSum[i - 1][j + length - 1]
37                 - prefixSum[i + length - 1][j - 1]
38                 + prefixSum[i - 1][j - 1];
39
40             if(sum <= threshold){
41                 return true;
42             }
43         }
44     }
45     return false;
46 }

```

隐式单调结构二分

典型例题：

[LeetCode 875 Koko Eating Bananas](#)

[LeetCode 410 Split Array Largest Sum](#)

875. 爱吃香蕉的珂珂

中等 相关标签 相关企业 Ag

珂珂喜欢吃香蕉。这里有 n 堆香蕉，第 i 堆中有 $piles[i]$ 根香蕉。警卫已经离开了，将在 h 小时后回来。

珂珂可以决定她吃香蕉的速度 k （单位：根/小时）。每个小时，她将会选择一堆香蕉，从中吃掉 k 根。如果这堆香蕉少于 k 根，她将吃掉这堆的所有香蕉，然后这一小时内不会再吃更多的香蕉。

珂珂喜欢慢慢吃，但仍然想在警卫回来前吃掉所有的香蕉。

返回她可以在 h 小时内吃掉所有香蕉的最小速度 k （ k 为整数）。

这类问题最为突出的特征就是单调的参数不明显，只要找到核心特性，便自然退化为第二类问题

本题单调对象：进食速度

题解：

```

3 public class MinEatingSpeeding_875 { 0个用法 新*
4     public int minEatingSpeed(int[] piles, int h) { 0个用法 新*
5         int min = 1, max = 0;
6         for(int pile : piles) {
7             max = Math.max(max, pile);
8         }
9         while(min <= max) {
10             int mid = (max + min) / 2 + min;
11             if(canFinishInTime(mid, piles, h)){
12                 max = mid - 1;
13             }else{
14                 min = mid + 1;
15             }
16         }
17         return min;
18     }
19
20     private boolean canFinishInTime(int speed, int[] piles, int h){ 1个用法 新*
21         long count = 0;
22         for(int pile : piles){
23             int coil = pile / speed;
24             count += (pile % speed == 0) ? coil : coil + 1;
25         }
26         return count <= h;
27     }
28 }
29

```

410. 分割数组的最大值

困难 相关标签 相关企业 Aa

给定一个非负整数数组 `nums` 和一个整数 `k`，你需要将这个数组分成 `k` 个非空的连续子数组，使得这 `k` 个子数组各自和的最大值 **最小**。

返回分割后最小的和的最大值。

子数组 是数组中连续的部分。

同理，不过多概述

本题单调对象：子数组容量

题解：

```

5 public class SplitArray_410 { 0个用法 新*
6     public int splitArray(int[] nums, int k) { 0个用法 新*
7         int min = 0, max = 0;
8
9         for(int num : nums){
10             max += num;
11             min = Math.max(min, num);
12         }
13         while(min <= max){
14             int mid = (max + min) / 2 + min;
15
16             if(checkIsPossible(mid, nums, k)){
17                 max = mid - 1;
18             }else{
19                 min = mid + 1;
20             }
21         }
22         return min;
23     }
24
25     private boolean checkIsPossible(int mid, int[] nums, int k){ 1个用法 新*
26         int count = 1, currSum = 0;
27
28         for(int num : nums){
29             if(currSum + num > mid){
30                 count ++;
31                 currSum = num;
32                 if(count > k) return false;
33             }else{
34                 currSum += num;
35             }
36         }
37         return true;
38     }
39 }
40

```